

Questionnaire Module

Author: Yasithzz | 2026

04 -- Questionnaire Module

LifeMate Project -- English Documentation

1. What is the Questionnaire Module?

The Questionnaire module is the entry point for all user-generated data in LifeMate. It is implemented as two React pages:

- * LifestylePage.jsx -- the daily lifestyle check-in form (8 inputs), holiday/leave calendar, and ML stress result display.
- * WellnessPage.jsx -- manual water intake and workout logging, plus personalised wellness suggestions driven by the latest ML stress prediction.

2. LifestylePage.jsx -- Line-by-Line Explanation

2.1 -- Imports and Constants

```
import { useEffect, useState } from 'react'
import { submitLifestyle, getLifestyleHistory, getLifestyleToday,
  getHolidays, addHoliday, removeHoliday, generateWeeklySchedule } from '../lib/api'
```

- * useState / useEffect -- React hooks for local state and side effects.
- * lib/api -- centralised API call functions that wrap fetch to the Spring Boot backend.

```
const MOODS = [
  { emoji: "😊", label: 'Great', value: 5 },
  { emoji: "🙂", label: 'Good', value: 4 },
  { emoji: "😐", label: 'Okay', value: 3 },
  { emoji: "😓", label: 'Low', value: 2 },
  { emoji: "😫", label: 'Stressed', value: 1 },
]

const WORKLOAD = ['Very Light','Light','Moderate','Heavy','Very Heavy']
const ENERGY = ['Exhausted','Tired','Okay','Energized','Very Energized']
const SOCIAL = ['None','Minimal','Moderate','Social','Very Social']
```

Label arrays that drive the button-based scale selectors. Each array index maps to the numeric value sent to the backend (index + 1 for 1-based scales, 0-based for mood by value field).

```
const STRESS_COLORS = {
  'Very Low': 'text-sky-400 bg-sky-500/15 border-sky-500/25',
  'High': 'text-orange-400 bg-orange-500/15 border-orange-500/25',
  'Very High': 'text-red-400 bg-red-500/15 border-red-500/25',
  ...
}
```

TailwindCSS class maps -- the ML result card changes colour based on the predicted stress level, giving instant visual feedback.

2.2 -- Component State

```
const [form, setForm] = useState({
  mood: 3, workload: 3, sleepHours: 7, energyLevel: 3,
  socialInteraction: 3, exerciseDone: 0, screenTimeHours: 4, waterCups: 6,
})
```

Holds the current values for all 8 form fields. Defaults correspond to mid-range values. Every button/stepper interaction calls `setForm(f => ({...f, fieldName: newValue}))` to update a single field immutably.

2.3 -- useEffect (Initial Data Load)

```
useEffect(() => {
  Promise.allSettled([getLifestyleToday(), getLifestyleHistory(), getHolidays()])
    .then(([t, h, hl]) => {
      if (t.status === 'fulfilled' && t.value) { setTodayEntry(t.value); setResult(t.value) }
      if (h.status === 'fulfilled') setHistory(h.value ?? [])
      if (hl.status === 'fulfilled') setHolidays(hl.value ?? [])
    })
}, [])
```

On mount, three API calls fire in parallel:

- * `getLifestyleToday()` -- checks if the user already submitted today; if so, shows the stored result immediately.
 - * `getLifestyleHistory()` -- loads the last 7 entries for the history list.
 - * `getHolidays()` -- loads the user's registered leave/holiday dates for the calendar.
-

2.4 -- handleSubmit()

```
const handleSubmit = async (e) => {
  e.preventDefault()
  setSubmitting(true)
  try {
    const r = await submitLifestyle({ mood, workload, sleepHours, ... })
    setResult(r)
    setTodayEntry(r)
    setHistory(prev => [r, ...prev.filter(e => e.id !== r.id)])
    localStorage.setItem('lm_schedule_needs_refresh', Date.now().toString())
  } finally { setSubmitting(false) }
}
```

What happens on Submit:

- * `e.preventDefault()` -- stops the browser from reloading the page.
- * `setSubmitting(true)` -- disables the button and shows "Analysing...".
- * `submitLifestyle()` -- POSTs the 8 values to `/api/lifestyle`. The backend calls the ML service, gets a stress prediction, and triggers schedule generation.
- * The returned object `r` contains `stressLevel`, `stressIndex`, and `timestamp`.
- * The ML result card renders and the history list is updated.
- * A `localStorage` timestamp signals the Schedule page to auto-refresh when visited next.

2.5 -- `handleRefreshSchedule()`

```
const handleRefreshSchedule = async () => {
  setRefreshing(true)
  try { await generateWeeklySchedule(result.stressLevel) }
  finally { setRefreshing(false) }
}
```

Manually regenerates the weekly schedule using the latest stress level -- useful if the user changed profile preferences and wants the schedule rebuilt without resubmitting lifestyle data.

2.6 -- Holiday Calendar -- Optimistic UI

```
const handleAddHoliday = async (date, type) => {
  setHolidays(prev => [...prev.filter(x => x.date !== date), { date, type, note: "" }])
  try {
    const h = await addHoliday(date, type)
    setHolidays(prev => prev.map(x => x.date === date ? h : x))
  } catch {
    setHolidays(prev => prev.filter(x => x.date !== date)) // rollback
  }
  refreshScheduleAfterHolidayChange()
}
```

Optimistic UI pattern: The calendar cell is coloured instantly (before the API call completes) so the

interface feels immediate. If the backend call fails, the local state is rolled back. After every holiday change, the weekly schedule is regenerated so the affected day appears as "Rest Day / Holiday".

The calendar supports three states per date: unmarked -> LEAVE -> HOLIDAY -> removed (cycled on click).

2.7 -- ScaleSelector Sub-component

```
function ScaleSelector({ value, onChange, labels }) {
  return (
    <div className="flex gap-1.5">
      {labels.map((label, i) => (
        <button key={i} type="button" onClick={() => onChange(i + 1)}
          className={`... ${value === i + 1 ? 'bg-violet-600/30 border-violet-500/50' : '...'}`}>
          {label}
        </button>
      ))}
    </div>
  )
}
```

A reusable button-row selector used for Workload, Energy Level, and Social Interaction. The selected option is highlighted with a violet glow. Clicking any button fires `onChange(i + 1)` where `i` is the 0-based array index.

2.8 -- Numeric Steppers (Sleep, Screen, Water)

```
<button onClick={() => setForm(f=>({...f, sleepHours: Math.max(0, parseFloat((f.sleepHours-.5).toFixed(1))))))}>
  <ChevronLeft />
</button>
<span>{form.sleepHours}</span>
<button onClick={() => setForm(f=>({...f, sleepHours: Math.min(12, parseFloat((f.sleepHours+.5).toFixed(1))))))}>
  <ChevronRight />
</button>
```

± stepper buttons increment/decrement in 0.5 steps within clamped min/max ranges. `toFixed(1)` prevents floating-point drift (e.g. 6.9000000001).

3. WellnessPage.jsx -- Line-by-Line Explanation

3.1 -- Water Intake Tracker

```
const handleLogWater = async (amt) => {
  await logWater(amt)
  await reload()
  setWaterFlash(true)
  setTimeout(() => setWaterFlash(false), 1500)
}
```

Clicking "+1 cup", "+2 cups", or "+3 cups" POSTs to /api/wellness/water, then reloads the totals. The brief green "Logged!" flash provides tactile confirmation.

Progress circle:

```
strokeDasharray={` ${pct * 2.639} 264`}
```

SVG circle circumference is 264 units (2×42). pct is $(\text{todayWater} / 8) * 100$. The stroke dash fills proportionally as water is logged toward the 8-cup daily goal.

3.2 -- Workout Logger

```
const handleLogWorkout = async () => {
  await logWorkout({ workoutType: workoutForm.workoutType,
    durationMinutes: parseInt(workoutForm.durationMinutes) })
  await reload()
  setWorkoutForm({ workoutType: 'Running', durationMinutes: '' })
}
```

Logs workout type and duration to the backend. Supported types: Running, Walking, Cycling, Swimming, Yoga, Weight Training, HIIT, Stretching, Other.

3.3 -- Personalised Suggestions (ML-driven)

```
(SUGGESTIONS_BY_LEVEL[stressLevel] ?? SUGGESTIONS_BY_LEVEL["Normal"])
  .map(({ type, icon, text, color }) => (
    <div className={`p-4 rounded-xl border ${color}`}>...</div>
  ))
```

The SUGGESTIONS_BY_LEVEL object holds 5-6 suggestions per stress tier. The page fetches the user's latest ML result (getLifestyleLatest()) and displays the matching suggestion set. If no lifestyle data has been submitted yet, it falls back to the "Normal" set.

Examples:

- * Very High stress -> "Stop & Rest", breathing techniques, gentle movement only.
- * Very Low stress -> High-intensity training, gratitude journaling, social connection.

4. Complete Data Flow -- Frontend -> Backend -> ML Model

User fills LifestylePage form (8 fields)

- ✓ Submit -> handleSubmit()
- ✓ POST /api/lifestyle (with JWT auth header)

Spring Boot Backend

- ✓ JWT token validated
- ✓ Lifestyle data saved to MongoDB (Lifestyle collection)
- ✓ POST http://localhost:5001/predict (8 features, no auth)

FastAPI ML Service (app.py)

- ✓ Pydantic validates ranges
- ✓ RandomForestClassifier.predict() -> stress_level integer (0-4)
- ✓ predict_proba() -> probability per class
- ✓ Response: { stress_level, stress_label, probabilities }

Spring Boot Backend

- ✓ Stress result attached to Lifestyle document (MongoDB update)
- ✓ WeeklyScheduleService.generateForCurrentWeek() called
- ✓ 7-day personalised schedule built and saved to MongoDB
- ✓ Response to frontend: { stressLevel, stressIndex, submittedAt, ... }

React Frontend

- ✓ ML result card shown (stress level + message + colour)
- ✓ localStorage signal set -> SchedulePage auto-refreshes
- ✓ WellnessPage: personalised suggestions update on next visit

Documentation -- LifeMate Project, Yasithzz, 2026